Kafka Stream Architecture

Friday, 19 June 2026 12:14 PM

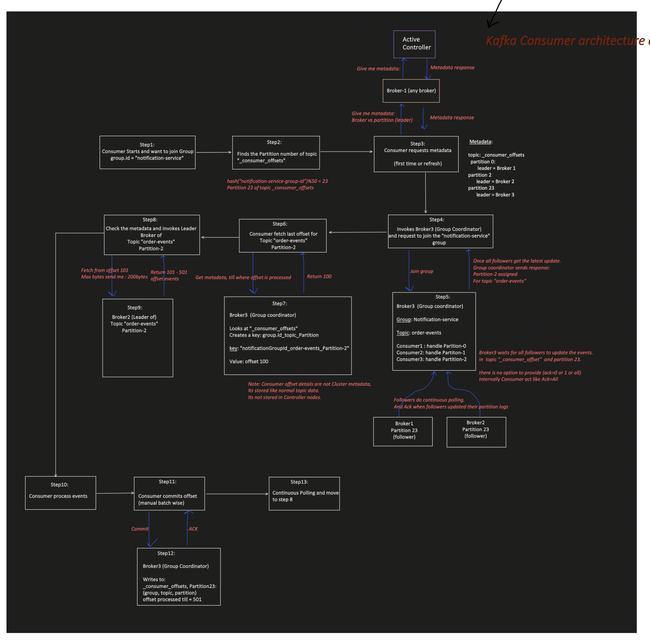
Print

#1 Pre-requisite:
Kafka Consumer Setup - Part1 | Setup and Internal working of Kafka Consumer with Demo



Kafka Streams reuses much of Kafka's Consumer architecture and underlying framework. So if we're already comfortable with how Kafka Consumers work, learning Kafka Streams becomes much easier.

Kafka Consumer architecture and framework flow



#2 Pre-requisite:
What is Kafka Stream and Need for Kafka Stream



Kafka Stream:

Stream:

In simple term, Stream has no start and no end, events keep arriving.

order-events:
[Order1] ----> [Order2] ----> [Order3] ----> [Order4] ----> . . . ----> [never ends]

Stream Processing:

Processing events as they arrive, continuously in real-time.

Kafka Steam

- is just a CLIENT LIBRARY for building stream processing application on top of kafka.
  - Its not a separate cluster or infrastructure.
  - It's a JAR file we add in our application.

Topology:

- It's a logical flow of how data moves and get processed from source Topics to sink Topics.
- This flow is like a DAG (Directed Acyclic Graph), data flow in one direction, no loops and cycles.

- We defined it once during application startup, after that Kafka Stream execute it forever.

Stateless Processing:

like raw material

Source:
Topic "order-events"

- *Entry point, read records from 1 or more Kafka Topics*
- *No parent processor*
- *Deserialize bytes to Key-value pairs*
- *Forward records to its child processors*

like
- *cut,*
- *paint,*
- *package etc.*

PROCESSORS

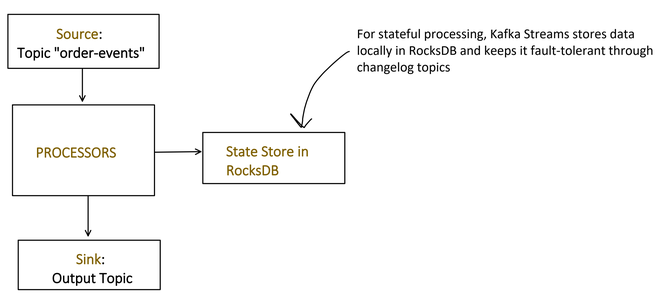
- *Business logic where transformation happens.*
- *Receive record from parent processor and process them*
- *Forward results to child processors*

like Shipping of finished material

Sink:
Output Topic

- *Exit point, writes records to a Kafka Topic*
- *No child processor*
- *Serialize key-value to bytes*
- *Write to output topic*

Stateful Processing:



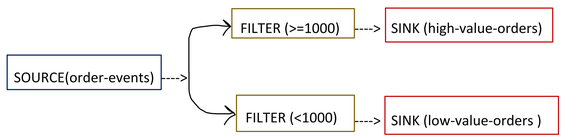
Different examples of Topology:

Print

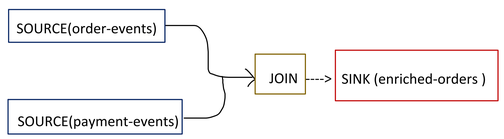
Simple linear Topology:



Branching Topology:



Multi-Source Join Topology:



Note:

At this point, lets not worry about:

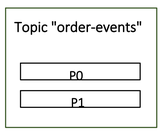
- *How to create source stream*
- *What are different processor like filter, map etc.*
- *How to put the result in sink stream*

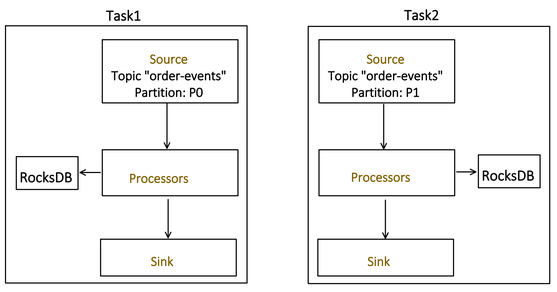
Etc.

Agenda here is to understand the architecture, in next part will cover the above topics.

TASK:

- It's a unit of parallelism.
- Each task has complete topology.
- No of tasks = no of Partitions
- Each task runs 1 partitions





Print

Each task has complete topology.

Actually this is not fully true, why?

Now lets go 1 level more deep:

Sub-Topology:

Sometimes we need re-partition. In that case 1 big topology is internally divided amount multiple sub topologies.

What re-partition means?

Lets understand with the below use case:

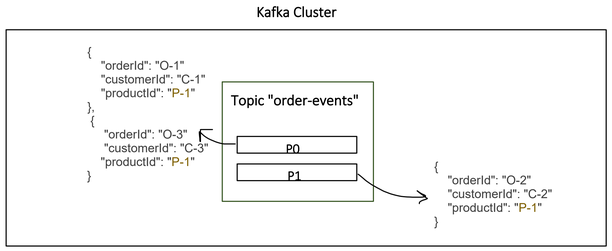
- Read order event

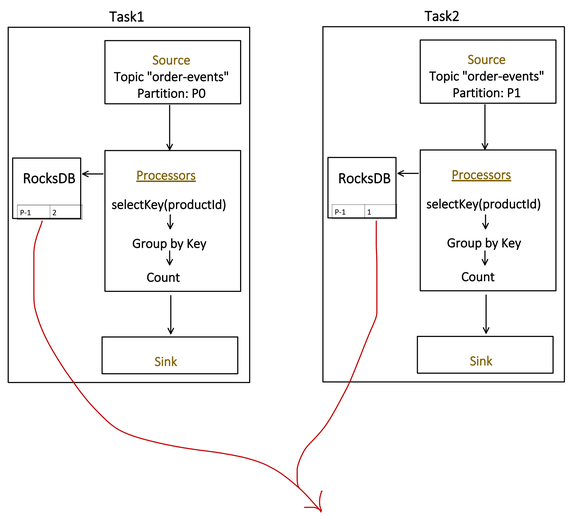
```
Key: orderId
Value: Order
        {
          "orderId":
          "customerId":
          "productId":
        }
```

- Need to find out, product wise order count
  Like:
  - P1 : 50 orders
  - P2 : 100 orders etc.

Problem: If there is no re-partition

Events are distributed across topic partitions.
So for example:
Product "P-1" orders are distributed across 2 partitions of Topic "order-events"

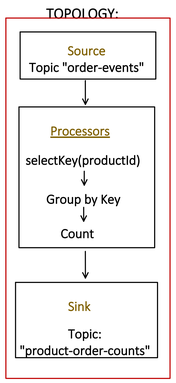


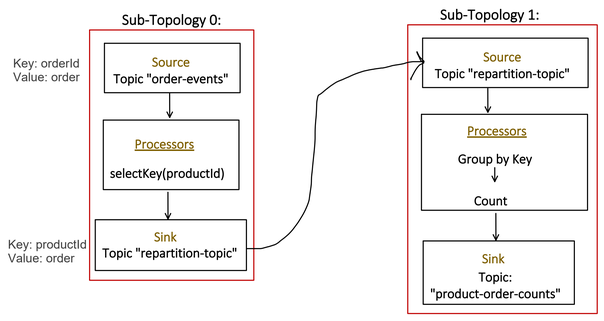


Count is distributed across multiple tasks

Print

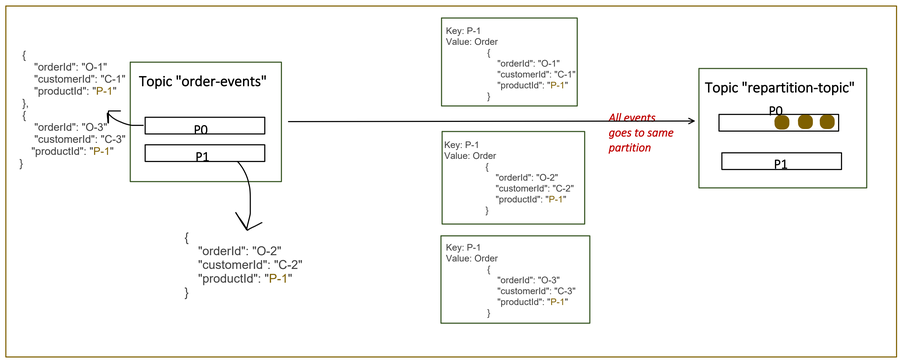
So we need re-partition based on Product ID.
So that all orders related to same product id goes to same partition and ultimately to same task.





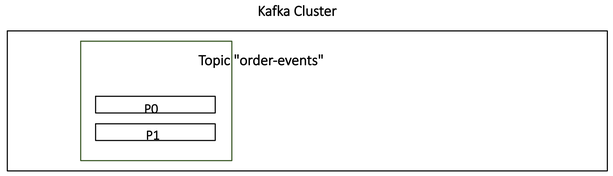
We just create TOPOLOGY, based on Processor (if repartition required) framework internally automatically creates REPARTITION topic and create Sub-topologies.

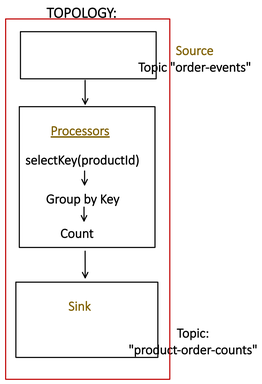
And configuration (like number of partitions, ISR etc) of Repartition topic is same as Source topic.

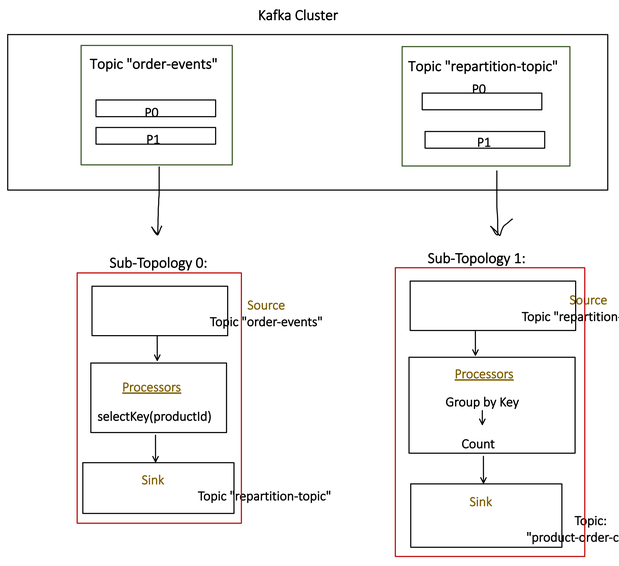


Now, lets come to TASK again:

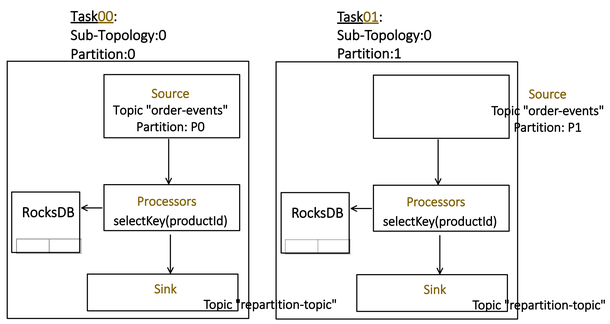
- It's a unit of parallelism.
- Each task has complete ~~topology~~ sub-topology for a specific partition.
- No of tasks = for each partition of a source topic in sub-topology, 1 task is created

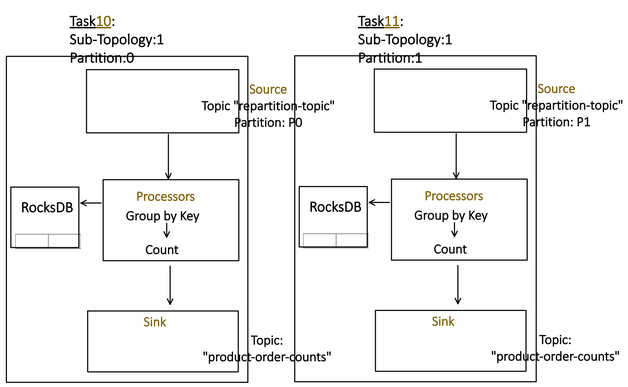




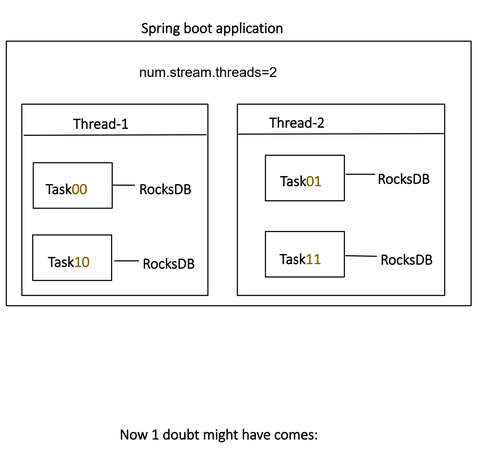


Total 4 tasks will be created





Now these 4 tasks will be distributed to our Stream application instances.



Now 1 doubt might have comes:

RocksDB is nothing but a Key-value file(database) store on Disk.
and
Each Kafka Task maintained its own RocksDB state.

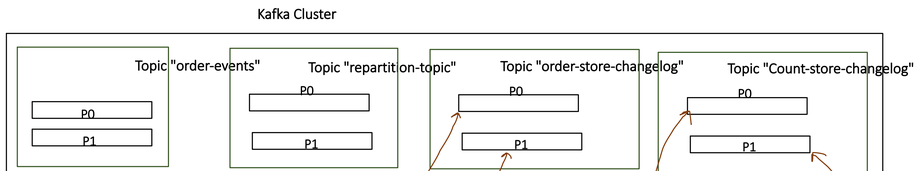
Now a doubt might have come is, what happens when machine dies or rebalance occurs?
Since RocksDB is local to a particular machine, the task may get assigned to another instance that does not have that local state.

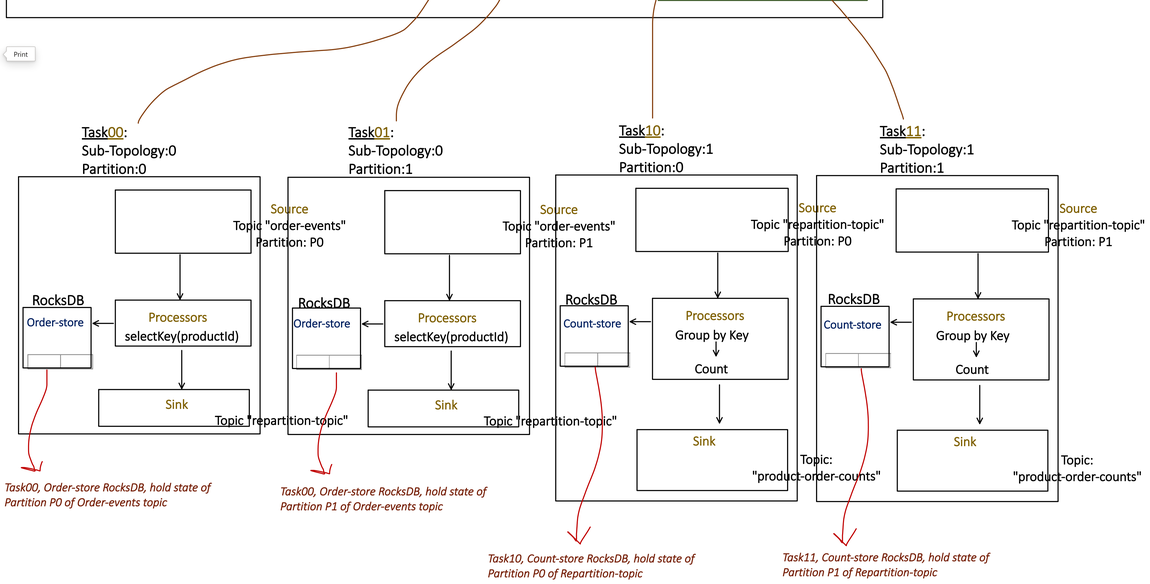
Then?

That's where changelog Topic comes into the picture.

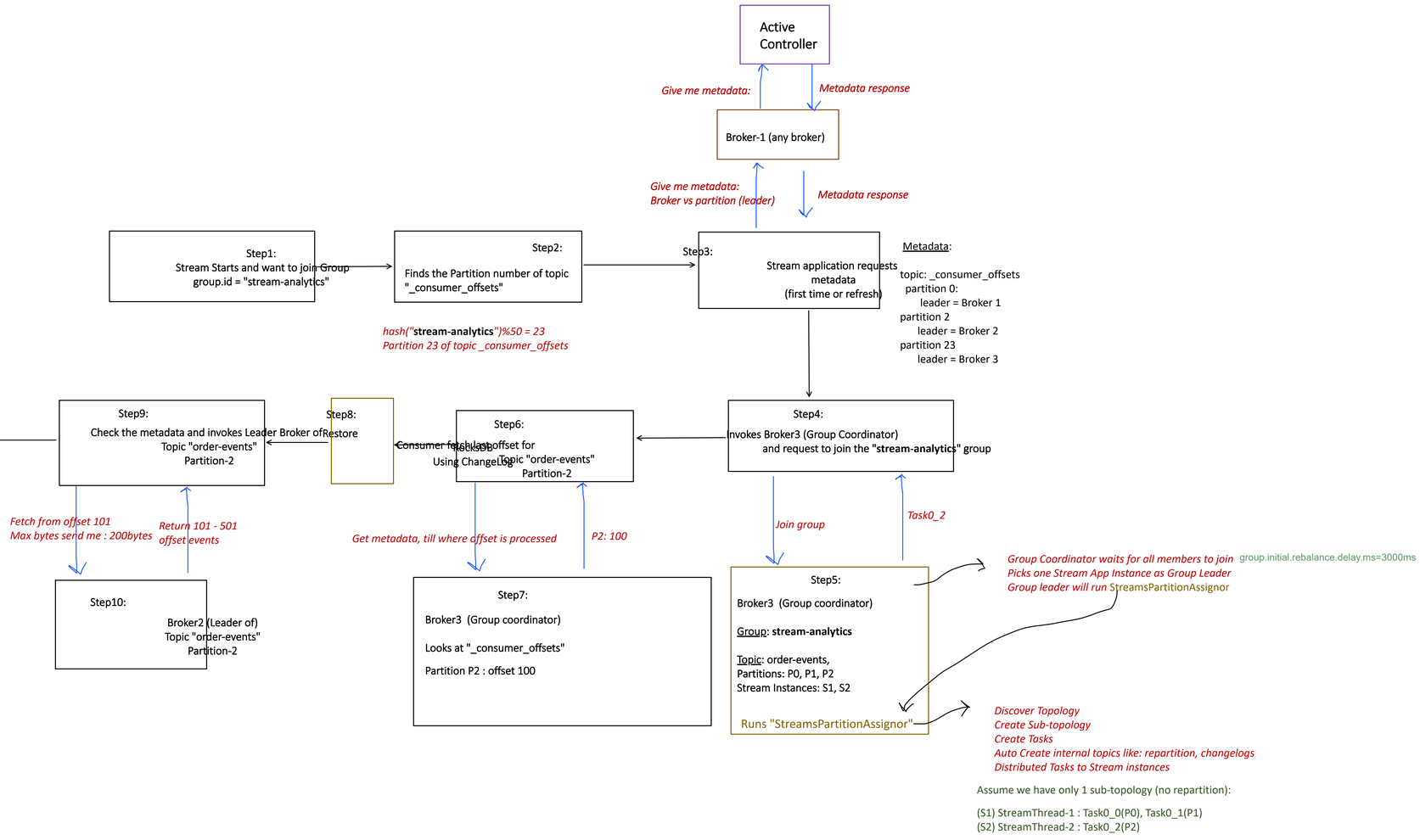
So, whenever a Task update its RocksDB. It also publish an event in changelog topic, this topic is internally created by Kafka Stream framework itself.

So when the task moves to another instance (due to rebalance) or original machine is lost, Kafka Stream create a new RocksDB store and restore its state by replaying records from the changelog topic.





Awesome, not we know the Architecture of Kafka Stream, lets see the complete flow (exactly same as Kafka Consumer)



Print

Step11:

Stream app process Task whichhas subtopology

Step12:

Stream app commits offset

Step14:

Continuous Polling and move to step 9

Commit

ACK

Step13:

Broker3 (Group Coordinator)

Writes to:
_consumer_offsets, Partition23:
(group, topic, partition)
offset processed till = 501

Group leader send this distribution info to Group Coordinator
Group Coordinator distributed to all members

Setup:

Dependency

```
<dependency>
<groupId>org.springframework.kafka</groupId>
<artifactId>spring-kafka</artifactId>
</dependency>

<dependency>
<groupId>org.apache.kafka</groupId>
<artifactId>kafka-streams</artifactId>
</dependency>
```

application.properties

```
server.port=8090
spring.application.name=kafka-streams-demo

spring.kafka.streams.application-id=order-
analytics

spring.kafka.streams.bootstrap-
servers=localhost:9092

spring.kafka.streams.properties[num.stream.thr
eads]=2

spring.kafka.streams.properties[state.dir]=/tmp
/kafka-streams
```

No of stream processing instances

Where RocksDB file will be stored on local disk

Enable Kafka Streams

```
@SpringBootApplication
@EnableKafkaStreams
public class KafkaStreamsDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(KafkaStreamsDemoApplicati
        on.class, args);
    }
```

}

Print

Define Topology

OrderStreamTopology.java (Configuration)

```
package com.eda.streams.config;
import com.eda.streams.model.Order;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Deserializer;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.common.serialization.Serializer;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.kstream.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OrderStreamTopology {
private static final ObjectMapper objectMapper = new
ObjectMapper();

@Bean
public void orderCountTopology(StreamsBuilder builder) {

        // SOURCE: read from "order-events" as raw JSON
        strings
        KStream<String, String> orderStream =
        builder.stream("order-events",
        Consumed.with(Serdes.String(), Serdes.String()));

        // FILTER: only orders with totalAmount > 100
        KStream<String, String> highValueOrders =
        orderStream.filter((key, value) -> {
        try {
                Order order = objectMapper.readValue(value,
                Order.class);
                return order.getTotalAmount() != null &&
                order.getTotalAmount() > 100;
                } catch (Exception e) {
                    return false;
                }
            });

        // Re-key by productId
        // This will trigger a REPARTITION topic
        KStream<String, String> productKeyedOrders =
                highValueOrders.selectKey((oldKey, value) -> {
                try {
                        Order order =
                        objectMapper.readValue(value,
                        Order.class);
                        return order.getProductId();
                    } catch (Exception e) {
                        return "UNKNOWN";
                    }
            });


        //RocksDB store name
        KTable<String, Long> productCounts =
                    productKeyedOrders
                .groupByKey(Grouped.with(
                        Serdes.String(),
                        Serdes.String()))
                .count(Materialized.as("product-count-store"));

        //output topic
        productCounts
                .toStream().mapValues(String::valueOf)
                .to("product-counts",
                Produced.with(Serdes.String(), Serdes.String()));

    }
}
```

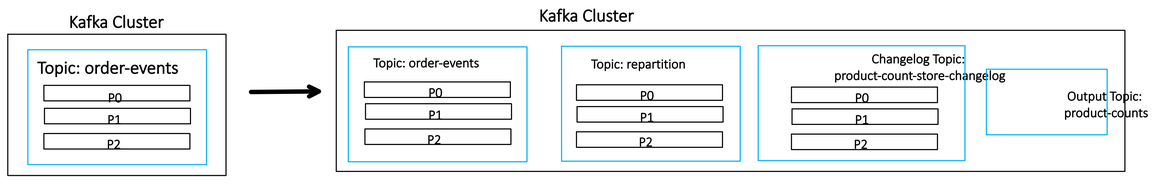
Order.java (POJO)

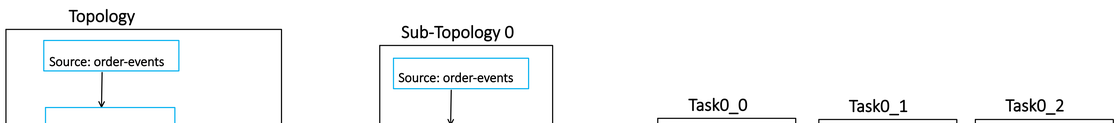
```
public class Order{
private String orderId;
private String customerId;
private String productId;
private Integer quantity;
private Double totalAmount;
private String status;

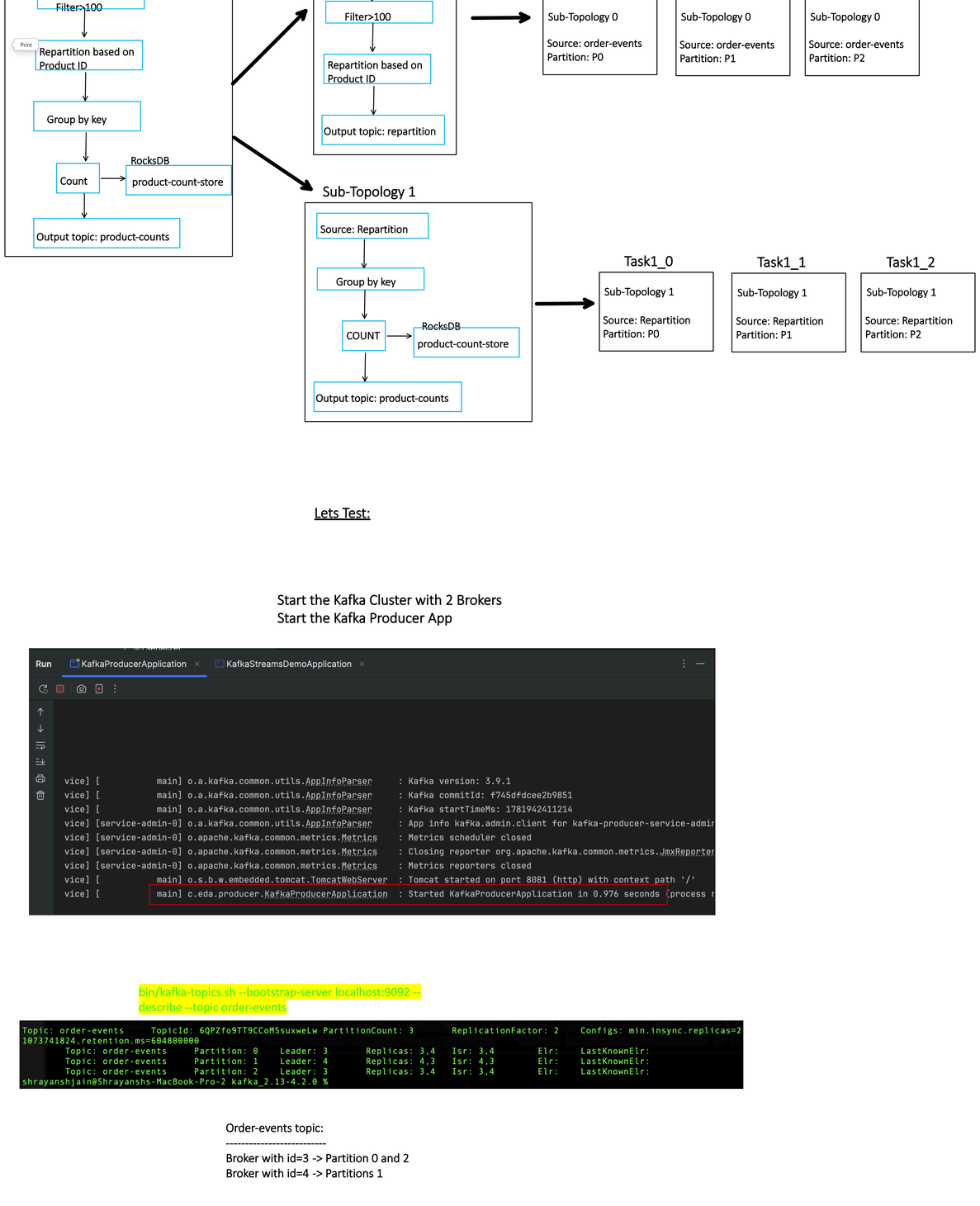
    //getters and setters

}
```

Expectation:







Lets Test:

Start the Kafka Cluster with 2 Brokers
Start the Kafka Producer App

bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic order-events

```
Topic: order-events     TopicId: 6QPZfo9TT9CCoMSsuxweLw PartitionCount: 3       ReplicationFactor: 2    Configs: min.insync.replicas=2
1073741824,retention.ms=604800000
        Topic: order-events     Partition: 0    Leader: 3       Replicas: 3,4   Isr: 3,4        Elr:    LastKnownElr:
        Topic: order-events     Partition: 1    Leader: 4       Replicas: 4,3   Isr: 4,3        Elr:    LastKnownElr:
        Topic: order-events     Partition: 2    Leader: 3       Replicas: 3,4   Isr: 3,4        Elr:    LastKnownElr:
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Order-events topic:

Broker with id=3 -> Partition 0 and 2
Broker with id=4 -> Partitions 1

Also validate what all topics present at this point of time:

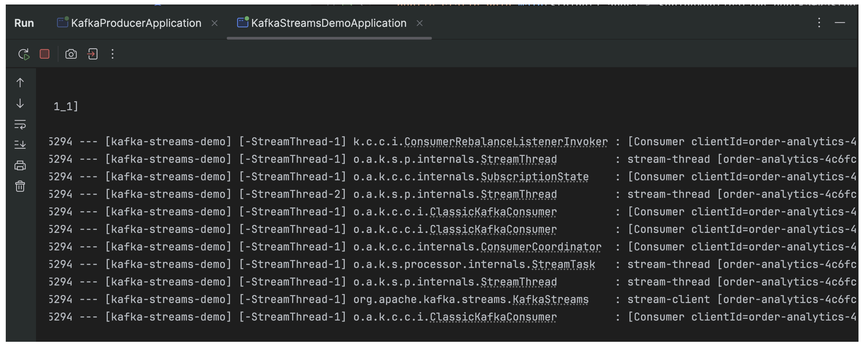
bin/kafka-topics.sh --bootstrap-server localhost:9092 --list

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
__consumer_offsets
order-events
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Only 2 topics present:
__consumer_offsets (internally get created)
Order-events (our producer app has created)

Print

Start the Kafka Stream Application



Also validate what all topics present at this point of time:

bin/kafka-topics.sh --bootstrap-server localhost:9092 --list

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
__consumer_offsets
order-analytics-product-count-store-changelog
order-analytics-product-count-store-repartition
order-events
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Only 4 topics present:
- __consumer_offsets (internally get created)
- order-events (our producer app has created)
- order-analytics-product-count-store-changelog (changelog topic internally get created)
- order-analytics-product-count-store-repartition(repartition topic internally get created)

Lets check which Task assigned to which Instance.

bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group order-analytics --describe

```
GROUP           TOPIC                                          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
order-analytics order-analytics-product-count-store-repartition 1          -               0               -               order-analytics-4c6fc4d3-b3b5-492c-ab88-caa80ccfaae7-StreamThread-2-consumer-fe04f48e-db72-410e-a532-5b9d31f68fc9
order-analytics order-analytics-product-count-store-repartition 2          -               0               -               order-analytics-4c6fc4d3-b3b5-492c-ab88-caa80ccfaae7-StreamThread-2-consumer-fe04f48e-db72-410e-a532-5b9d31f68fc9
order-analytics order-events                                   1          -               0               -               order-analytics-4c6fc4d3-b3b5-492c-ab88-caa80ccfaae7-StreamThread-2-consumer-fe04f48e-db72-410e-a532-5b9d31f68fc9
order-analytics order-analytics-product-count-store-repartition 0          -               0               -               order-analytics-4c6fc4d3-b3b5-492c-ab88-caa80ccfaae7-StreamThread-1-consumer-453df86a-7dd1-4be7-b804-2bd74c76f5a2
order-analytics order-events                                   0          -               0               -               order-analytics-4c6fc4d3-b3b5-492c-ab88-caa80ccfaae7-StreamThread-1-consumer-453df86a-7dd1-4be7-b804-2bd74c76f5a2
order-analytics order-events                                   2          -               0               -               order-analytics-4c6fc4d3-b3b5-492c-ab88-caa80ccfaae7-StreamThread-1-consumer-453df86a-7dd1-4be7-b804-2bd74c76f5a2
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

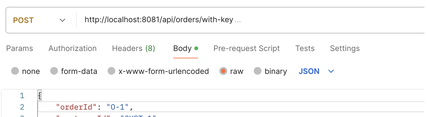
Thread1 owns Task:

- order-events P0
- order-events P2
- order-analytics-product-count-store-repartition P0

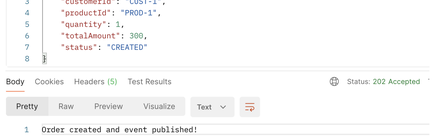
Thread2 owns Task:

- order-events P1
- order-analytics-product-count-store-repartition P1
- order-analytics-product-count-store-repartition P2

Lets publish an event:



Print



Producer application logs:

```
2026-06-20T14:00:48.918+05:30  INFO 53795 --- [kafka-producer-service] [ad | producer-1]
2026-06-20T14:00:48.918+05:30  INFO 53795 --- [kafka-producer-service] [ad | producer-1]
Order event sent successfully!
Partition: 0
Offset: 0
```

Partition P0 leader is Broker1 with id=3

bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker1-logs/order-events-0/00000000000000000000.log

Lets order-events Topic, key is order ID

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker1-logs/order-events-0/0
00000000000000000000.log
Dumping /tmp/broker1-logs/order-events-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1002 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isCont
rol: false deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1781944248918 size: 239 magic: 2 compresscodec: snappy crc: 284709179 isvalid: true
| offset: 0 CreateTime: 1781944248918 keySize: 3 valueSize: 112 sequence: 0 headerKeys: [__TypeId__] key: O-1 payload: {"orderId":"O-1","customerId":"CUST-1
","productId":"PROD-1","quantity":1,"totalAmount":300.0,"status":"CREATED"}
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Lets check Repartition Topic, key is now PRODUCT ID

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker1-logs/order-analytics-
product-count-store-repartition-0/00000000000000000000.log
Dumping /tmp/broker1-logs/order-analytics-product-count-store-repartition-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 2 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl
: false deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1781944248918 size: 228 magic: 2 compresscodec: none crc: 1164846927 isvalid: true
| offset: 0 CreateTime: 1781944248918 keySize: 6 valueSize: 112 sequence: 0 headerKeys: [__TypeId__] key: PROD-1 payload: {"orderId":"O-1","customerId":"CUS
T-1","productId":"PROD-1","quantity":1,"totalAmount":300.0,"status":"CREATED"}
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Now lets check RocksDB local

Each Tasks maintain its own local RocksDB state.

T

Now lets check changelog topic

bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker1-logs/order-analytics-product-count-store-changelog-0/00000000000000000000.log

Print

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker1-logs/order-analytics-
product-count-store-changelog-0/00000000000000000000.log
Dumping /tmp/broker1-logs/order-analytics-product-count-store-changelog-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 2 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl
: false deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1781944248918 size: 82 magic: 2 compresscodec: none crc: 1921361186 isvalid: true
| offset: 0 CreateTime: 1781944248918 keySize: 6 valueSize: 8 sequence: 0 headerKeys: [] key: PROD-1 payload:
```

Now lets check output topic:

bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker2-logs/product-counts-0/00000000000000000000.log

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log --files /tmp/broker2-logs/product-counts-0
/00000000000000000000.log
Dumping /tmp/broker2-logs/product-counts-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 2 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl
: false deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1781944248918 size: 115 magic: 2 compresscodec: none crc: 1597431120 isvalid: true
| offset: 0 CreateTime: 1781944248918 keySize: 6 valueSize: 1 sequence: 0 headerKeys: [__TypeId__] key: PROD-1 payload: 2
```